Technical Brief: Local Vector Architecture & RAG Pipelines

Systems Optimization & Semantic Coordinate Routing

---

1. Mathematical Core: Embedding Models

An embedding model turns human language into high-dimensional vector coordinates $\mathbb{R}^d$. Unlike keyword-based parsers that match exact text, an embedding network uses self-attention to place concepts into spatial regions where meaning determines distance.

Given an input string, the model outputs a normalized floating-point array with a fixed dimension ($d = 768$ for `nomic-embed-text`). Relationships between vectors are computed using **Cosine Similarity**, which measures the angle between two vectors:

$$\text{Similarity} = \cos(\theta) = \frac{\mathbb{A} \cdot \mathbb{B}}{\|\mathbb{A}\| \, \|\mathbb{B}\|}$$

When $\theta \approx 0$, the vectors point in the same direction, which means their meanings are close even if they use different words.

2. Local Storage Layer: pgvector

Standard relational databases struggle with high-dimensional nearest-neighbor searches. `pgvector` extends PostgreSQL to natively store, index, and query these vector arrays using special operators.

It supports Approximate Nearest Neighbor (ANN) searches with **HNSW (Hierarchical Navigable Small World)** graphs or **IVFFlat** inverted files, avoiding slow $O(N)$ full table scans. Instead of matching strings, the database runs spatial distance calculations directly, finding matching rows using the cosine distance operator (`<=>`).

3. Architecture of the Local RAG Pipeline

Retrieval-Augmented Generation (RAG) reduces hallucinations by injecting verified information into the model's context window.

Pipeline Steps:

1. **Data Ingestion:** Text or PDF files are read into strings.
2. **Context Chunking:** Text is split using a sliding window (`chunk_size=500`, `chunk_overlap=100`) so context carries across boundaries.
3. **Spatial Encoding:** Chunks are sent to `nomic-embed-text` via Ollama to generate vector coordinates, then stacked in memory with NumPy.
4. **Query Mapping & Distance Scan:** The input query is converted to a vector. A distance search finds the top-k matching chunks.
5. **Context Isolation & Chat Inference:** The matched chunks are placed into a system prompt and sent to `llama3.2` to stream a response.

4. Ecosystem Synergy Matrix

The components work together in an integrated data processing loop:

Component	What It Does	Pipeline Role
	Manages local model endpoints and handles concurrency for the encoder and generative model.	System Execution Interface

Nomic Embed	Converts text into fixed-width vector coordinates ($\mathbb{R}^{768}$).	Deterministic Math Compiler
NumPy Matrix / pgvector	Stores raw vectors and runs fast distance calculations using linear algebra.	Spatial Storage & Search Engine
Llama 3.2	Processes structured context and generates human-readable responses.	Inference Generation Layer

5. Python Implementation

The project includes a `LocalRAGPipeline` class in Python that implements this architecture. It ingests text or PDF files, splits them into overlapping chunks, generates embeddings locally via Ollama's `nomic-embed-text`, and stores them in a NumPy matrix. On query, it converts the prompt to a vector, finds the top-k most similar chunks by cosine similarity, and sends them as context to `llama3.2` for generation.